

UNIT-5
LEETCODEEXERCISE

1.PRODUCT SALES ANALYSIS I

PROBLEM SUMMARY

Table: Sales

Column Name	Type
sale_id	int
product_id	int
year	int
quantity	int
price	int

Table: Product

Column Name	Type
product_id	int
product_name	varchar

product_id is the primary key of this table.

TASK:

Write an SQL query that reports the product_name, year, and price for each sale_id in the Sales table.

SOLUTION:

MySQL   Auto

```
1 # Write your MySQL query statement below
2 select p.product_name, s.year, s.price
3 from Sales s
4 left join Product p
5 on s.product_id = p.product_id
```

Output

```
| product_name | year | price |
| ----- | ---- | ----- |
| Nokia      | 2008 | 5000  |
| Nokia      | 2009 | 5000  |
| Apple       | 2011 | 9000  |
```

Expected

```
| product_name | year | price |
| ----- | ---- | ----- |
| Nokia      | 2009 | 5000  |
| Nokia      | 2008 | 5000  |
| Apple       | 2011 | 9000  |
```

2. PRODUCT PRICE AT A GIVEN DATE:

PROBLEM SUMMARY

Table: Products

Column Name	Type
product_id	int
new_price	int
change_date	date

TASK

Write a solution to find the prices of all products on the date 2019-08-16. Return the result table in any order.

SOLUTION:

MySQL   Auto

```
1 # Write your MySQL query statement below
2 WITH LatestPrice AS (
3     SELECT
4         product_id,
5         new_price,
6         ROW_NUMBER() OVER (
7             PARTITION BY product_id
8             ORDER BY change_date DESC
9         ) AS rn
10    FROM Products
11   WHERE change_date <= '2019-08-16'
12 )
13
14 SELECT
15     p.product_id,
16     COALESCE(lp.new_price, 10) AS price
17 FROM
18     (SELECT DISTINCT product_id FROM Products) p
19 LEFT JOIN LatestPrice lp
20     ON p.product_id = lp.product_id
21    AND lp.rn = 1
```

Output

product_id	price
1	35
2	50
3	10

Expected

product_id	price
1	35
2	50
3	10

3. IMMEDIATE FOOD DELIVERY II

PROBLEM SUMMARY:

Table: Delivery

Column Name	Type
delivery_id	int
customer_id	int
order_date	date

customer_pref_delivery_date	date
-----------------------------	------

TASK

Write a solution to find the percentage of immediate orders in the first orders of all customers, rounded to 2 decimal places.

SOLUTION:

```
MySQL v Auto
1 # Write your MySQL query statement below
2 WITH FirstOrders AS (
3     SELECT *,
4         ROW_NUMBER() OVER (
5             PARTITION BY customer_id
6             ORDER BY order_date
7         ) AS rn
8     FROM Delivery
9 )
10
11 SELECT
12     ROUND(
13         100.0 *
14         SUM(CASE
15             WHEN order_date = customer_pref_delivery_date
16             THEN 1
17             ELSE 0
18         END)
19         / COUNT(*),
20         2
21     ) AS immediate_percentage
22 FROM FirstOrders
23 WHERE rn = 1
```

Output

```
| immediate_percentage |
| ----- |
| 50                |
```

Expected

```
| immediate_percentage |
| ----- |
| 50                |
```

4. MONTHLY TRANSACTION I

PROBLEM SUMMARY

Table: Transactions

Column Name	Type
id	int
country	varchar
state	enum
amount	int
trans_date	date

TASK

Write an SQL query to find for each month and country, the number of transactions and their total amount, the number of approved transactions and their total amount.

SOLUTION:

MySQL   Auto

```
1 # Write your MySQL query statement below
2
3 SELECT
4     DATE_FORMAT(trans_date, '%Y-%m') AS month,
5     country,
6     COUNT(*) AS trans_count,
7     SUM(CASE
8         WHEN state = 'approved' THEN 1
9         ELSE 0
10    END) AS approved_count,
11    SUM(amount) AS trans_total_amount,
12    SUM(CASE
13        WHEN state = 'approved' THEN amount
14        ELSE 0
15    END) AS approved_total_amount
16 FROM Transactions
17 GROUP BY DATE_FORMAT(trans_date, '%Y-%m'), country;
```

Output

month	country	trans_count	approved_count	trans_total_amount	approved_total_amount
2018-12	US	2	1	3000	1000
2019-01	US	1	1	2000	2000
2019-01	DE	1	1	2000	2000

Expected

month	country	trans_count	approved_count	trans_total_amount	approved_total_amount
2018-12	US	2	1	3000	1000
2019-01	US	1	1	2000	2000
2019-01	DE	1	1	2000	2000

5. LAST PERSON TO FIT IN THE BUS

PROBLEM SUMMARY

Table: Queue

Column Name	Type
person_id	int
person_name	varchar
weight	int
turn	int

TASK

Write a solution to find the person_name of the last person that can fit on the bus without exceeding the weight limit. The test cases are generated such that the first person does not exceed the weight limit.

SOLUTION:

MySQL   Auto

```
1 # Write your MySQL query statement below
2 WITH RunningTotal AS (
3     SELECT
4         person_name,
5         turn,
6         SUM(weight) OVER (ORDER BY turn) AS total_weight
7     FROM Queue
8 )
9
10 SELECT person_name
11 FROM RunningTotal
12 WHERE total_weight <= 1000
13 ORDER BY turn DESC
14 LIMIT 1;
```

Output

```
| person_name |  
| ----- |  
| John Cena  |
```

Expected

```
| person_name |  
| ----- |  
| John Cena  |
```